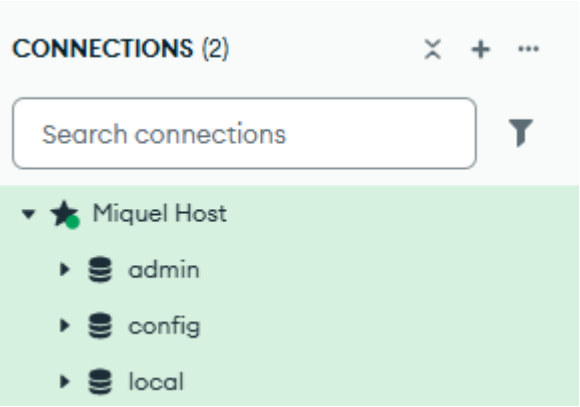
Autor: Miquel Cerdà Cifre – Tasca BD 7

GITHUB: https://github.com/mcc-fp/DAW-BD-MiquelCerdaCifre/tree/main/BD07
LOOM: https://www.loom.com/share/480c5fb047e041aca0b4e1d70f23f3dd

NOTE: To be able to do the exercises with MongoDB Shell:

1- I already créate a connection



2-) Open a new command Windows and start MongoDB with: mongod

```
Símbolo del sistema - mongo
Microsoft Windows [Versión 10.0.26200.8037]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\mnora>mongod
{"t":{"$date":"2026-04-13T21:20:41.786+02:00"},"s":"I",  "c":"-",        "id":8991200, "ctx":"thread1","msg":"Shuffling initializers","attr":{"seed":1776725029}}
{"t":{"$date":"2026-04-13T21:20:41.850+02:00"},"s":"I",  "c":"CONTROL",  "id":97374,   "ctx":"thread1","msg":"Automatically disabling TLS 1.0 and TLS 1.1, to force-enable TLS 1.1 specify --sslDisabledProtocols
 'TLS1_0'; to force-enable TLS 1.0 specify --sslDisabledProtocols 'none'"}
{"t":{"$date":"2026-04-13T21:20:41.854+02:00"},"s":"I",  "c":"NETWORK",  "id":4915701, "ctx":"thread1","msg":"Initialized wire specification","attr":{"spec":{"incomingExternalClient":{"minWireVersion":0,"maxWi
reVersion":27},"incomingInternalClient":{"minWireVersion":0,"maxWireVersion":27},"outgoing":{"minWireVersion":6,"maxWireVersion":27},"isInternalClient":true}}}
{"t":{"$date":"2026-04-13T21:20:41.854+02:00"},"s":"I",  "c":"CONTROL",  "id":5945603, "ctx":"thread1","msg":"Multi threading initialized"}
{"t":{"$date":"2026-04-13T21:20:41.854+02:00"},"s":"I",  "c":"CONTROL",  "id":4615611, "ctx":"initandlisten","msg":"MongoDB starting","attr":{"pid":26548,"port":27017,"dbPath":"/data/db","architecture":"64-bit
","host":"PCLAPTOP"}}
{"t":{"$date":"2026-04-13T21:20:41.854+02:00"},"s":"I",  "c":"CONTROL",  "id":23398,   "ctx":"initandlisten","msg":"Target operating system minimum version","attr":{"targetMinOS":"Windows 7/Windows Server 2008
 R2"}}
```

3-) In another command window start Mongodb Shell with: mongosh

```
mongosh mongodb://127.0.0.
Microsoft Windows [Versión 10.0.26200.8037]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\mnora>mongosh
Current Mongosh Log ID: 69dd42316a89fea022b5f898
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2
.5.0
Using MongoDB:          8.2.6
Using Mongosh:          2.5.0
mongosh 2.8.2 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-13T21:20:42.490+02:00: Access control is not enabled for the database. Read and write access to data and conf
iguration is unrestricted
   2026-04-13T21:20:42.490+02:00: This server is bound to localhost. Remote systems will be unable to connect to this se
rver. Start the server with --bind_ip <address> to specify which IP addresses it should serve responses from, or with --
bind_ip_all to bind to all interfaces. If this behavior is desired, start the server with --bind_ip 127.0.0.1 to disable
 this warning
------

test> showdbs
ReferenceError: showdbs is not defined
test> show dbs
admin   40.00 KiB
config  36.00 KiB
local   80.00 KiB
```

Write the code used to perform the following tasks from the MongoDB shell:

1. Create a database called zoo.

   use zoo

```
test> showdbs
ReferenceError: showdbs is not defined
test> show dbs
admin   40.00 KiB
config  36.00 KiB
local   80.00 KiB
test> use zoo
switched to db zoo
```

   This command switches to the zoo database. If it does not exist yet, MongoDB will create it when data is inserted. Is important to say that databse will not créate it until we add a new collection.

2. Insert a collection called animals

   db.createCollection("animals")

```
zoo> db.createCollection("animals")
{ ok: 1 }
```

3. Confirm that the database and the collection have been successfully created.

show dbs	–>To be able to show the databases we have
use zoo	-> To use the zoo databse
show collections	-> To be able to show the collections we have inside zoo

4. Insert the documents contained in the animals.json file.

db.animals.insertMany([copy/paste the animals.json file content])

```
acknowledged: true,
insertedIds: {
  '0': 1,
  '1': 2,
  '2': 3,
  '3': 4,
  '4': 5,
  '5': 6,
  '6': 7,
  '7': 8,
  '8': 9,
  '9': 10,
  '10': 11,
  '11': 12,
  '12': 13,
  '13': 14,
  '14': 15,
  '15': 16,
  '16': 17,
  '17': 18,
  '18': 19,
  '19': 20,
  '20': 21,
  '21': 22,
  '22': 23,
  '23': 24,
  '24': 25,
  '25': 26,
  '26': 27,
  '27': 28,
  '28': 29,
  '29': 30,
  '30': 31,
```

....

As there are many documents in the json file I need to envolve with the charácters [ ] the contento f the json file.

5- Write a MongoDB query to find animals that have Rainforest as their habitat and are attended by a caretaker named Emily Brown.

```
db.animals.find({
    habitat: "Rainforest",
    "caretakers.nameCaretaker": "Emily Brown"
})
```

```
zoo> db.animals.find({
...   habitat: "Rainforest",
...   "caretakers.nameCaretaker": "Emily Brown"
... })
[
  {
    _id: 2,
    name: 'Rajah',
    age: 8,
    gender: 'Male',
    species: 'Tiger',
    habitat: 'Rainforest',
    birthplace: 'Bandipur Tiger Reserve, India',
    parents: [
      { name: 'Shere Khan', gender: 'Male' },
      { name: 'Tala', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 102, nameCaretaker: 'Emily Brown' } ]
  }
]
```

6- Write a MongoDB query to display only the name, species, and habitat of all animals sorted by their species in ascending order.

```
db.animals.find(
{},
{
   _id: 0,
   name: 1,
   species: 1,
   habitat: 1
}
).sort({ species: 1 }) we use 1 for ascending order, for descending order will be -1
```

```
zoo> db.animals.find(
...   {},
...   {
...     _id: 0,
...     name: 1,
...     species: 1,
...     habitat: 1
...   }
... ).sort({ species: 1 })
[
  { name: 'Balu', species: 'Bear', habitat: 'Mountain Forest' },
  { name: 'Koda', species: 'Bear', habitat: 'Mountain Forest' },
  { name: 'Kenai', species: 'Bear', habitat: 'Mountain Forest' },
  { name: 'Smokey', species: 'Bear', habitat: 'Mountain Forest' },
  { name: 'Yogi', species: 'Bear', habitat: 'Mountain Forest' },
  { name: 'Buffy', species: 'Buffalo', habitat: 'Grasslands' },
  { name: 'Bison', species: 'Buffalo', habitat: 'Grasslands' },
  { name: 'Grazor', species: 'Buffalo', habitat: 'Grasslands' },
  { name: 'Shadow', species: 'Coyote', habitat: 'Desert' },
  { name: 'Maya', species: 'Coyote', habitat: 'Desert' },
  { name: 'Blaze', species: 'Coyote', habitat: 'Desert' },
  { name: 'Storm', species: 'Coyote', habitat: 'Desert' },
  { name: 'Luna', species: 'Coyote', habitat: 'Desert' },
  { name: 'Max', species: 'Coyote', habitat: 'Desert' },
  { name: 'Sandy', species: 'Dromedary', habitat: 'Desert' },
  { name: 'Hump', species: 'Dromedary', habitat: 'Desert' },
  { name: 'Dune', species: 'Dromedary', habitat: 'Desert' },
  { name: 'Liberty', species: 'Eagle', habitat: 'Mountains' },
  { name: 'Freedom', species: 'Eagle', habitat: 'Mountains' },
  { name: 'Tantor', species: 'Elephant', habitat: 'Grasslands' }
]
Type "it" for more
zoo>
```

7 .Write a MongoDB query to find animals with the species Tiger and display only their name and age, sorted by age in descending order.

```
db.animals.find(
{ species: "Tiger" },
{
    _id: 0,
    name: 1,
    age: 1
}
).sort({ age: -1 })
```

```
mongosh mongodb://127.0.0.

zoo> db.animals.find(
...   { species: "Tiger" },
...   {
...     _id: 0,
...     name: 1,
...     age: 1
...   }
... ).sort({ age: -1 })
[
  { name: 'Shere Khan', age: 10 },
  { name: 'Shere Khan', age: 10 },
  { name: 'Tigris', age: 9 },
  { name: 'Rajah', age: 8 },
  { name: 'Rajah', age: 8 },
  { name: 'Tala', age: 7 },
  { name: 'Tala', age: 7 },
  { name: 'Kumba', age: 6 },
  { name: 'Zara', age: 5 },
  { name: 'Kiran', age: 4 }
]
```

This query filters animals by species Tiger, displays only name and age, and sorts them from oldest to youngest.

We use -1 for descending order

8. Write a MongoDB query to insert the next animal into the animals collection:

name: "Leo",

age: 7,

gender: "Male",

species: "Lion",

habitat: "Savanna",

birthplace: "Masai Mara, Kenya",

parents: "Unknown", gender: "Male", Unknown", gender: "Female"

caretakers: id: 501, name: "Jake Adams" }]

```
db.animals.insertOne({
name: "Leo",
age: 7,
gender: "Male",
species: "Lion",
habitat: "Savanna",
birthplace: "Masai Mara, Kenya",
parents: [
{ name: "Unknown", gender: "Male" },
{ name: "Unknown", gender: "Female" }
],
caretakers: [
{ idCaretaker: 501, nameCaretaker: "Jake Adams" }
]
})
```

```
zoo> db.animals.insertOne({
...   name: "Leo",
...   age: 7,
...   gender: "Male",
...   species: "Lion",
...   habitat: "Savanna",
...   birthplace: "Masai Mara, Kenya",
...   parents: [
...     { name: "Unknown", gender: "Male" },
...     { name: "Unknown", gender: "Female" }
...   ],
...   caretakers: [
...     { idCaretaker: 501, nameCaretaker: "Jake Adams" }
...   ]
... })
{
  acknowledged: true,
  insertedId: ObjectId('69dd4d2e6a89fea022b5f899')
}
```

acknowledged: true -> meaning the insertion was ok

9. Write a MongoDB query to update the age (to 8) of the animal you just inserted using the $set operator

```
db.animals.updateOne(
    { name: "Leo", species: "Lion" },
    { $set: { age: 8 } }
)
```

```
zoo> db.animals.updateOne(
...       { name: "Leo", species: "Lion" },
...       { $set: { age: 8 } }
... )
...
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

I use the $set operator to be able to update the age to 8

10. Write a MongoDB query to rename the habitat field to environment for all animals in the animals collection.

```
db.animals.updateMany(
   {},
   { $rename: { "habitat": "environment" } }
)
```

```
zoo> db.animals.updateMany(
...    {},
...    { $rename: { "habitat": "environment" } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 71,
  modifiedCount: 71,
  upsertedCount: 0
}
```

11- Write a MongoDB query using the aggregation framework to find animals aged les than 10 and sort them alphabetically by their name.

```
db.animals.aggregate([
{
    $match: { age: { $lt: 10 } }
},
{
    $sort: { name: 1 }
}
])
```

```
zoo> db.animals.aggregate([
... {
...     $match: { age: { $lt: 10 } }
... },
... {
...     $sort: { name: 1 }
... }
... ])
...
[
  {
    _id: 22,
    name: 'Bagheera',
    age: 8,
    gender: 'Male',
    species: 'Panther',
    birthplace: 'Amazon Rainforest, Brazil',
    parents: [
      { name: 'Unknown', gender: 'Male' },
      { name: 'Unknown', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 406, nameCaretaker: 'Alice Green' } ],
    environment: 'Rainforest'
  },
  {
    _id: 55,
    name: 'Bison',
    age: 9,
    gender: 'Male',
    species: 'Buffalo',
    birthplace: 'Serengeti National Park, Tanzania',
    parents: [
      { name: 'Unknown', gender: 'Male' },
      { name: 'Unknown', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 524, nameCaretaker: 'Nathan Hall' } ],
    environment: 'Grasslands'
  },
  {
    _id: 49,
    name: 'Blaze',
    age: 7,
    gender: 'Male',
    species: 'Coyote',
    birthplace: 'Nevada, USA',
    parents: [
      { name: 'Max', gender: 'Male' },
      { name: 'Luna', gender: 'Female' }
```

12. Write a MongoDB query using the aggregation framework to group animals by their species and count the number of animals in each species.

```
db.animals.aggregate([
{
    $group: {
    _id: "$species",
   totalAnimals: { $sum: 1 }
}
},
{
  $sort: { _id: 1 }
}
])
```

This aggregation groups documents by species and counts how many animals belong to each species.

```
zoo> db.animals.aggregate([
...   {
...     $group: {
...       _id: "$species",
...       totalAnimals: { $sum: 1 }
...     }
...   },
...   {
...     $sort: { _id: 1 }
...   }
... ])
[
  { _id: 'Bear', totalAnimals: 5 },
  { _id: 'Buffalo', totalAnimals: 3 },
  { _id: 'Coyote', totalAnimals: 6 },
  { _id: 'Dromedary', totalAnimals: 3 },
  { _id: 'Eagle', totalAnimals: 2 },
  { _id: 'Elephant', totalAnimals: 4 },
  { _id: 'Giraffe', totalAnimals: 4 },
  { _id: 'Hippopotamus', totalAnimals: 3 },
  { _id: 'Kangaroo', totalAnimals: 3 },
  { _id: 'Lion', totalAnimals: 6 },
  { _id: 'Mandrill', totalAnimals: 7 },
  { _id: 'Panther', totalAnimals: 3 },
  { _id: 'Rattlesnake', totalAnimals: 1 },
  { _id: 'Rhinoceros', totalAnimals: 1 },
  { _id: 'Spider Monkey', totalAnimals: 3 },
  { _id: 'Tiger', totalAnimals: 10 },
  { _id: 'Wolf', totalAnimals: 7 }
]
```

13. Write a MongoDB query to create a new collection called Caretakers by extracting all caretakers from the animals collection. Ensure that caretakers do not repeat and include an array of the animals they attend to.

```
db.animals.aggregate([
 {
  $unwind: "$caretakers"
 },
 {
  $group: {
   _id: "$caretakers.idCaretaker",
   nameCaretaker: { $first: "$caretakers.nameCaretaker" },
   animals: { $addToSet: "$name" }
  }
 },
 {
  $project: {
   _id: 0,
   idCaretaker: "$_id",
   nameCaretaker: 1,
   animals: 1
  }
 },
 {
  $out: "Caretakers"
 }
])
```

```
zoo> db.animals.aggregate([
...   {
...     $unwind: "$caretakers"
...   },
...   {
...     $group: {
...       _id: "$caretakers.idCaretaker",
...       nameCaretaker: { $first: "$caretakers.nameCaretaker" },
...       animals: { $addToSet: "$name" }
...     }
...   },
...   {
...     $project: {
...       _id: 0,
...       idCaretaker: "$_id",
...       nameCaretaker: 1,
...       animals: 1
...     }
...   },
...   {
...     $out: "Caretakers"
...   }
... ])

zoo> show collections
animals
Caretakers
Habitats
zoo>
```

14. Write a MongoDB query to find the top 3 oldest animals.

db.animals.find().sort({ age: -1 }).limit(3)

This query sorts all animals by age in descending order and returns only the first 3 documents.

```
zoo> db.animals.find().sort({ age: -1 }).limit(3)
[
  {
    _id: 14,
    name: 'Jumbo',
    age: 30,
    gender: 'Male',
    species: 'Elephant',
    birthplace: 'Kruger National Park, South Africa',
    parents: [
      { name: 'Dumbo', gender: 'Male' },
      { name: 'Ella', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 302, nameCaretaker: 'Emily Brown' } ],
    environment: 'Grasslands'
  },
  {
    _id: 13,
    name: 'Dumbo',
    age: 25,
    gender: 'Male',
    species: 'Elephant',
    birthplace: 'Serengeti National Park, Tanzania',
    parents: [
      { name: 'Unknown', gender: 'Male' },
      { name: 'Unknown', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 301, nameCaretaker: 'John Smith' } ],
    environment: 'Grasslands'
  },
  {
    _id: 15,
    name: 'Ella',
    age: 20,
    gender: 'Female',
    species: 'Elephant',
    birthplace: 'Serengeti National Park, Tanzania',
    parents: [
      { name: 'Unknown', gender: 'Male' },
      { name: 'Unknown', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 303, nameCaretaker: 'Alice Green' } ],
    environment: 'Grasslands'
  }
]
```

15.Write a MongoDB query to display animals between the 6th and 12th entries in the collection based on their insertion order.

db.animals.find().sort({ _id: 1 }).skip(5).limit(7)

This query sorts the documents by _id in ascending order to preserve the insertion sequence, skips the first 5 animals, and returns the next 7 animals, which correspond to the 6th through the 12th entries.

```
zoo> db.animals.find().sort({ _id: 1 }).skip(5).limit(7)
[
  {
    _id: 6,
    name: 'Zara',
    age: 5,
    gender: 'Female',
    species: 'Tiger',
    birthplace: 'Sundarbans, India',
    parents: [
      { name: 'Rajah', gender: 'Male' },
      { name: 'Tala', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 106, nameCaretaker: 'Nathan Green' } ],
    environment: 'Rainforest'
  },
  {
    _id: 7,
    name: 'Kiran',
    age: 4,
    gender: 'Male',
    species: 'Tiger',
    birthplace: 'Chitwan National Park, Nepal',
    parents: [
      { name: 'Kumba', gender: 'Male' },
      { name: 'Tigris', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 107, nameCaretaker: 'Michael Brown' } ],
    environment: 'Rainforest'
  },
  {
    _id: 8,
    name: 'Mufasa',
    age: 10,
    gender: 'Male',
    species: 'Lion',
    birthplace: 'Serengeti National Park, Tanzania',
    parents: [
      { name: 'Unknown', gender: 'Male' },
      { name: 'Unknown', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 201, nameCaretaker: 'John Smith' } ],
    environment: 'Savanna'
  },
```

16. Write a MongoDB query using the aggregation framework to create a new collection called Habitats. This collection should group animals by their environment (formerly habitat) and include the following fields.

- The environment name.
- The total number of animals in that environment.
- An array of unique species in that environment, sorted alphabetically.
- An array of unique caretakers who attend animals in that environment.

```
db.animals.aggregate([
 {
  $group: {
   _id: "$environment",
   animalIds: { $addToSet: "$_id" },
   species: { $addToSet: "$species" },
   caretakersArrays: { $push: "$caretakers" }
  }
 },
 {
  $project: {
   _id: 0,
   environment: "$_id",
   totalAnimals: { $size: "$animalIds" },
   species: {
    $sortArray: {
     input: "$species",
     sortBy: 1
    }
   },
   caretakersArrays: 1
  }
 },
 {
  $unwind: "$caretakersArrays"
 },
 {
```

```
    $unwind: "$caretakersArrays"
  },
  {
    $group: {
      _id: "$environment",
      totalAnimals: { $first: "$totalAnimals" },
      species: { $first: "$species" },
      caretakers: { $addToSet: "$caretakersArrays.nameCaretaker" }
    }
  },
  {
    $project: {
      _id: 0,
      environment: "$_id",
      totalAnimals: 1,
      species: 1,
      caretakers: 1
    }
  },
  {
    $out: "Habitats"
  }
])
```

To be able to check as the $out goes to Habitats and not to the screen:

```
db.Habitats.find().pretty()
```

```
zoo> db.Habitats.find().pretty()
[
  {
    _id: ObjectId('69de9003d1f4bcd0b5e472c3'),
    totalAnimals: 5,
    species: [ 'Bear' ],
    caretakers: [
      'Sophia Green',
      'Jake Adams',
      'Emily Brown',
      'John Smith',
      'Michael Davis'
    ],
    environment: 'Mountain Forest'
  },
  {
    _id: ObjectId('69de9003d1f4bcd0b5e472c4'),
    totalAnimals: 23,
    species: [ 'Mandrill', 'Panther', 'Spider Monkey', 'Tiger' ],
    caretakers: [
      'Alice Green',   'Michael Brown',
      'Mark Taylor',   'John Smith',
      'Emily Brown',   'Nathan Green',
      'Sophia White',  'Olivia Harris',
      'Sophia Taylor', 'Mark Brown',
      'Olivia Davis',  'Jake Adams',
      'Emily Green'
    ],
    environment: 'Rainforest'
  },
  {
    _id: ObjectId('69de9003d1f4bcd0b5e472c5'),
    totalAnimals: 2,
    species: [ 'Eagle' ],
    caretakers: [ 'Emily Brown', 'John White' ],
    environment: 'Mountains'
  },
  {
    _id: ObjectId('69de9003d1f4bcd0b5e472c6'),
    totalAnimals: 3,
    species: [ 'Hippopotamus' ],
    caretakers: [ 'Sophia Taylor', 'Alice Green', 'Mark Brown' ],
    environment: 'River'
  },
```